人工智能与深度学习

机器学习（三）

华中科技大学 机械科学与工程学院
陈伟

2026

聚类
(Clustering)

聚类任务

- 在无监督学习中，训练样本的标记信息是未知的，学习目标是揭示数据内在的性质及规律

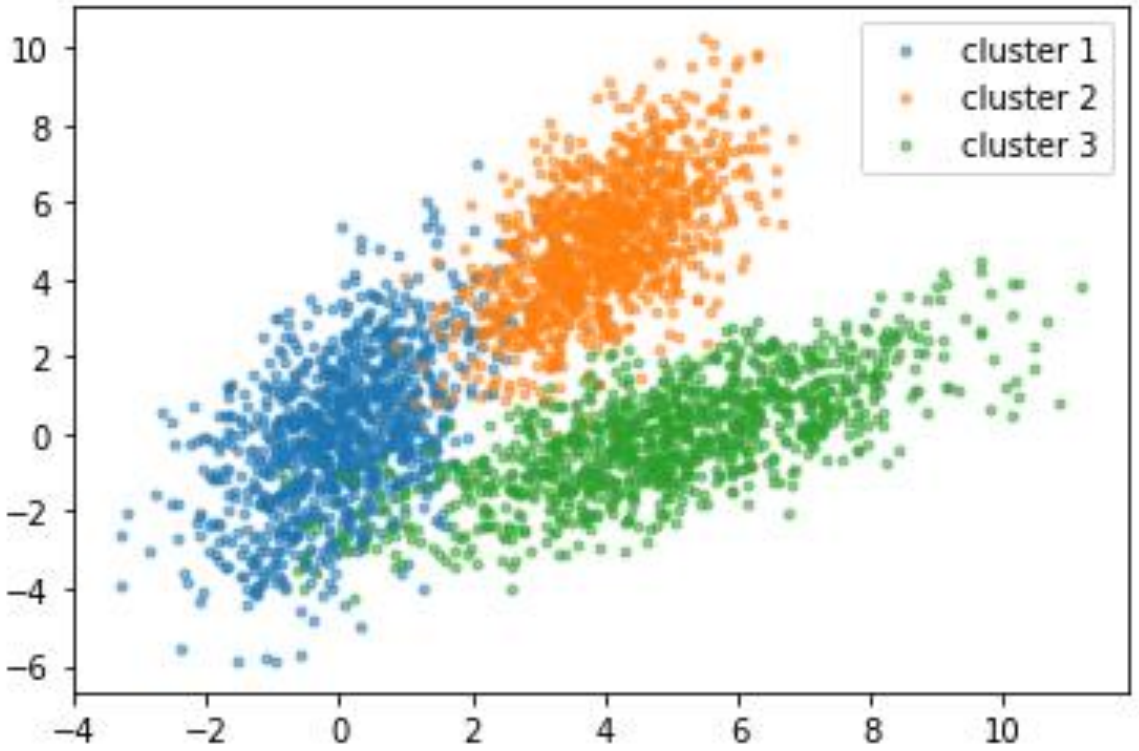
- 此类任务中最常见的、研究最多、应用最广的是“聚类”

聚类任务

- **聚类试图将数据集中的样本划分为若干个通常是不相交的子集，每个子集称为一个“簇”（cluster）**

- **每个簇可能对应一些潜在的概念（类别），如有故障/无故障，猫/狗等**
  - 需要说明的是，这些概念对于聚类算法而言事先是未知的，聚类过程仅能自动形成簇，而簇所对应的概念语义则需要由使用者来把握和命名

聚类任务



聚类任务的数学描述

- 假定样本集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_m\}$ 包含 m 个无标签样本，每个样本 $\boldsymbol{x}_i = [x_{i1}, x_{i2}, ..., x_{in}]^T$ 是一个 n 维特征向量

  聚类算法将 D 划分为 k 个不相交的簇 $\{C_l | l = 1, 2, ..., k\}$，$C_{l'} \cap_{l' \neq l} C_l = \emptyset$，且 $D = \cup_{l=1}^{k} C_l$

  相应地，我们用 $\lambda_j \in \{1, 2, ..., k\}$ 表示样本 $\boldsymbol{x}_j$ 的“簇标签”（cluster label），即 $\boldsymbol{x}_j \in C_{\lambda_j}$

聚类的性能度量

- **如何衡量聚类结果的好坏?**
  - 简单说，“簇内相似度”高，且“簇间相似度”低

- **聚类性能度量大致有两类**
  - 外部指标：将聚类结果与某个“参考模型”进行比较
  - 内部指标：直接考察聚类结果，而不利用任何参考模型

类别	核心思想	是否需要真实标签	适用场景	常见指标
外部指标	与“标准答案”对比	是	算法验证、有标签数据	ARI, AMI, 同质性/完整性
内部指标	评估簇内紧密和簇间分离	否	实际应用的主流、无标签数据	轮廓系数, DBI, CH指数

聚类的性能度量：内部指标

- 考虑聚类结果的簇划分 $\mathcal{C} = \{C_1, C_2, ..., C_k\}$，定义

$$\text{avg}(C) = \frac{2}{|C|(|C|-1)} \sum_{1 \le i < j \le |C|} \text{dist}(\boldsymbol{x}_i, \boldsymbol{x}_j),$$

其中，$\text{dist}(\cdot, \cdot)$ 用于计算两个样本之间的距离；$\text{avg}(C)$ 表示了簇 C 内任意两样本间的平均距离；

$$\text{diam}(C) = \max_{1 \le i < j \le |C|} \text{dist}(\boldsymbol{x}_i, \boldsymbol{x}_j),$$

$\text{diam}(C)$ 是簇 C 内样本间的最远距离

聚类的性能度量：内部指标

- 定义

$$d_{\min}(C_i, C_j) = \min_{\boldsymbol{x}_i \in C_i, \boldsymbol{x}_j \in C_j} \operatorname{dist}(\boldsymbol{x}_i, \boldsymbol{x}_j),$$

$d_{\min}(C_i, C_j)$ 是簇 C_i 和簇 C_j 最近样本间的距离；

$$d_{\text{cen}}(C_i, C_j) = \operatorname{dist}(\boldsymbol{\mu}_i, \boldsymbol{\mu}_j),$$

其中 $\boldsymbol{\mu}$ 代表簇 C 的中心点，$\boldsymbol{\mu} = \frac{1}{C} \sum_{1 \le i \le |C|} \boldsymbol{x}_i$；
$d_{\text{cen}}(C_i, C_j)$ 是簇 C_i 和簇 C_j 中心点间的距离

聚类的性能度量：内部指标

- 基于以上这些定义，可以提出下面这些常用的聚类性能度量内部指标

  - DB指数（Davies-Bouldin Index, DBI）

$$\mathrm{DBI} = \frac{1}{k}\sum_{i=1}^{k}\max_{j\neq i}\left(\frac{\mathrm{avg}(C_i)+\mathrm{avg}(C_j)}{d_{\mathrm{cen}}(C_i,C_j)}\right)$$

  - Dunn指数（Dunn Index，DI）

$$\mathrm{DI} = \min_{1\leq i\leq k}\left[\min_{j\neq i}\left(\frac{d_{\min}(C_i,C_j)}{\max_{1\leq l\leq k}\mathrm{diam}(C_l)}\right)\right]$$

DBI越小越好，DI越大越好

聚类中的距离计算

- 给定两个样本（两个向量） $\boldsymbol{x}_i = [x_{i1}, ..., x_{in}]$ 和 $\boldsymbol{x}_j = [x_{j1}, ..., x_{jn}]$，“闵可夫斯基距离”（Minkowski distance）定义为

$$\mathrm{dist}_{\mathrm{mk}}(\boldsymbol{x}_i, \boldsymbol{x}_j) = \left(\sum_{u=1}^{n} |x_{iu} - x_{ju}|^p \right)^{\frac{1}{p}}$$

闵可夫斯基距离即为 $\boldsymbol{x}_i - \boldsymbol{x}_j$ 的 L_p 范数 $||\boldsymbol{x}_i - \boldsymbol{x}_j||_p$

聚类中的距离计算

- $p = 2$ 时，闵可夫斯基距离即欧氏距离（Euclidean distance）

$$\mathrm{dist}_{\mathrm{ed}}(\boldsymbol{x}_i, \boldsymbol{x}_j) = ||\boldsymbol{x}_i - \boldsymbol{x}_j||_2 = \sqrt{\sum_{u=1}^{n} |x_{iu} - x_{ju}|^2}$$

$p = 1$ 时，闵可夫斯基距离即曼哈顿距离（Manhattan distance）

$$\mathrm{dist}_{\mathrm{man}}(\boldsymbol{x}_i, \boldsymbol{x}_j) = ||\boldsymbol{x}_i - \boldsymbol{x}_j||_1 = \sum_{u=1}^{n} |x_{iu} - x_{ju}|$$

聚类中的距离计算

- 蓝线：欧氏距离；红线：曼哈顿距离



聚类中的距离计算：离散特征

- **离散特征有的是“有序的”，有的则是“无序的”**
  - 有序的：{1037号，1039号，1041号}，{二等座，一等座，商务座}，{1，2，3}
  - 无序的：{飞机，火车，轮船}，{机械学院，建筑学院，医学院}

- **闵可夫斯基距离的概念可用于有序特征，但一般不适用于无序特征**

聚类中的距离计算

- 当样本空间中不同属性的重要性不同时，可使用“加权距离”（weighted distance）

  加权闵可夫斯基距离：

$$\mathrm{dist}_{\mathrm{wmk}}(\boldsymbol{x}_i, \boldsymbol{x}_j) = (w_1 |x_{i1} - x_{j1}|^p + \ldots + w_n |x_{in} - x_{jn}|^p)^{\frac{1}{p}}$$

K 均值（k-means）算法

- **K 均值算法是最常用的聚类算法之一**

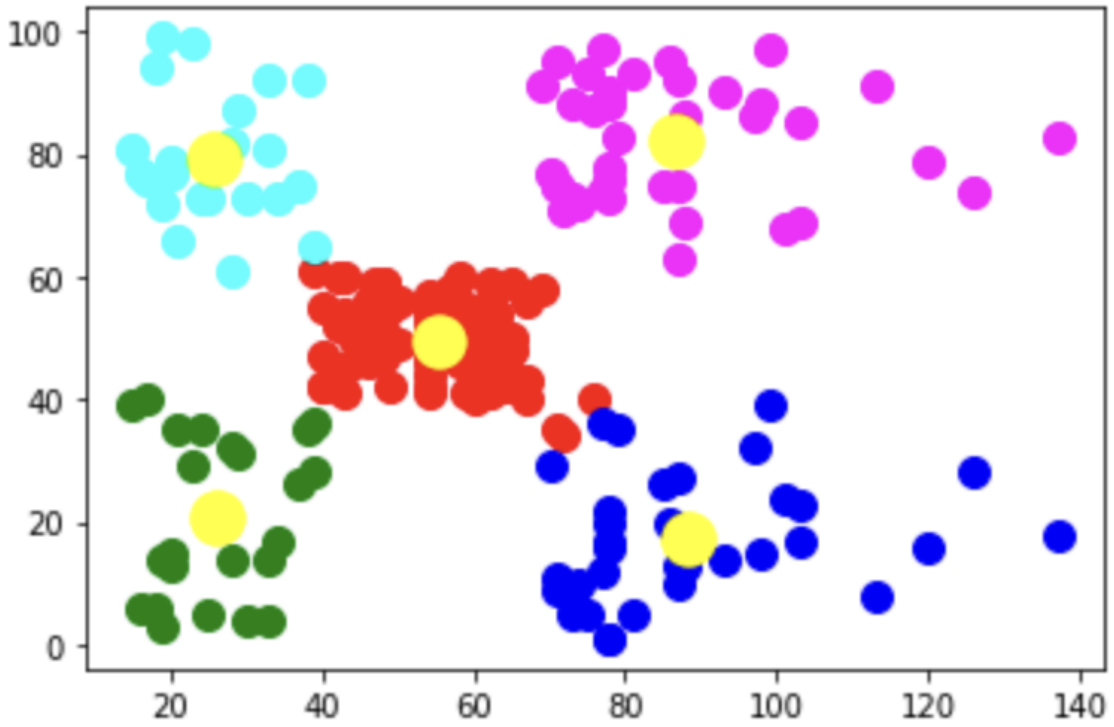
- 给定样本集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_m\}$，k均值算法针对聚类所得簇划分 $\mathcal{C} = \{C_1, C_2, ..., C_k\}$ 最小化平方误差

$$E = \sum_{i=1}^{k} \sum_{\boldsymbol{x} \in C_i} ||\boldsymbol{x} - \boldsymbol{\mu}_i||_2^2,$$

其中 $\boldsymbol{\mu}_i = \frac{1}{|C_i|} \sum_{\boldsymbol{x} \in C_i} \boldsymbol{x}$ 是簇 C_i 的均值向量，k 是一个预先给定的值

K 均值算法

- $$E = \sum_{i=1}^{k} \sum_{\boldsymbol{x} \in C_i} ||\boldsymbol{x} - \boldsymbol{\mu}_i||_2^2,$$



E 一定程度上刻画了簇内样本围绕簇均值的紧密程度；*E* 值越小则簇内样本相似度越高

K 均值算法

- 求解最优的 k 均值聚类结果并不容易

- 实际的 k 均值算法一般采用贪心策略，通过迭代优化来近似求解

K 均值算法：示例

- **在scikit-learn中**

创建聚类模型 + 训练模型，两行合并为一行

Examples

```
>>> from sklearn.cluster import KMeans
>>> import numpy as np
>>> X = np.array([[1, 2], [1, 4], [1, 0],
...               [10, 2], [10, 4], [10, 0]])
>>> kmeans = KMeans(n_clusters=2, random_state=0).fit(X)
>>> kmeans.labels_
array([1, 1, 1, 0, 0, 0], dtype=int32)
>>> kmeans.predict([[0, 0], [12, 3]])
array([1, 0], dtype=int32)
>>> kmeans.cluster_centers_
array([[10.,  2.],
       [ 1.,  2.]])
```

集成学习
(Ensemble Learning)

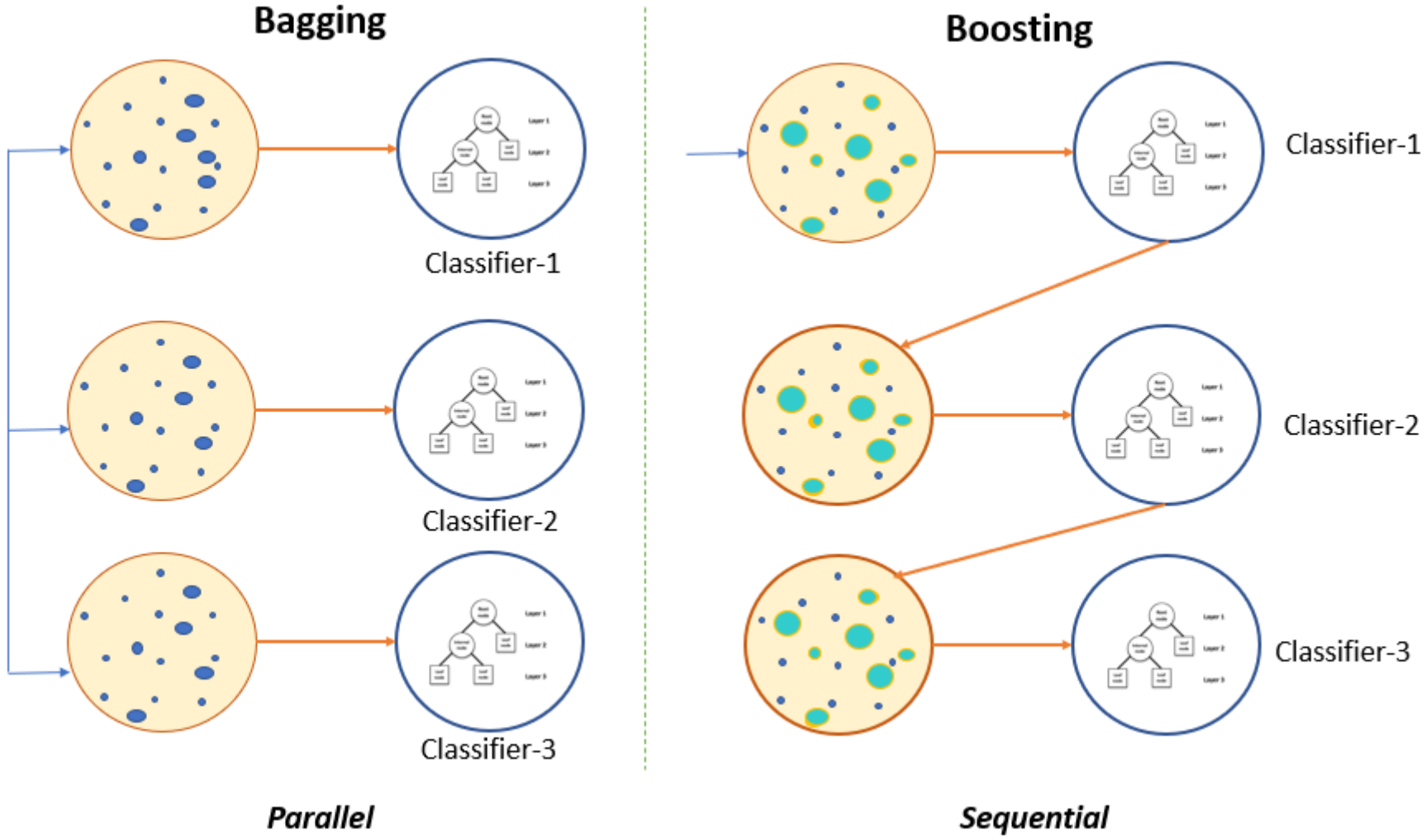
集成学习

- 集成学习通过构建并结合多个机器学习模型来完成学习任务

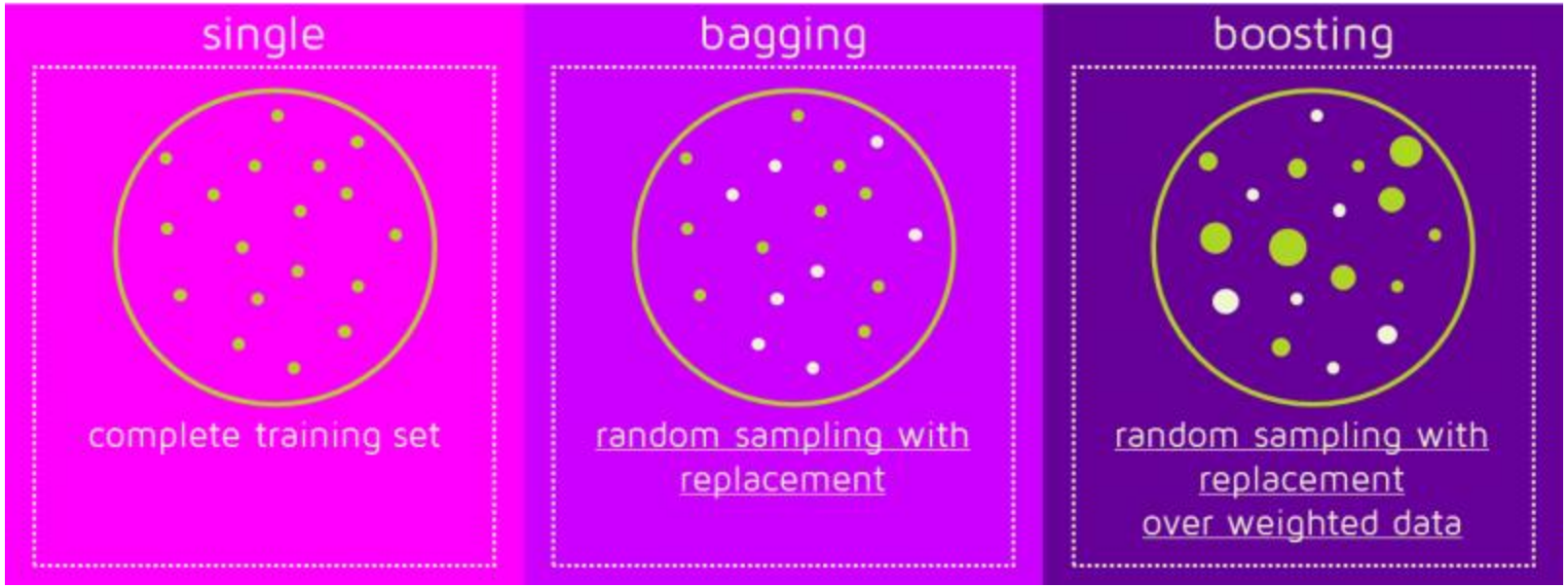
- Bagging：“随机有放回抽样得到多组数据；训练多个模型，每个模型输出一个结果；各模型一起投票决定最终结果”

- Boosting：“训练出模型后，把错的部分拿出来重点强化，继续训练，得到多个模型，再加权结合”

集成学习

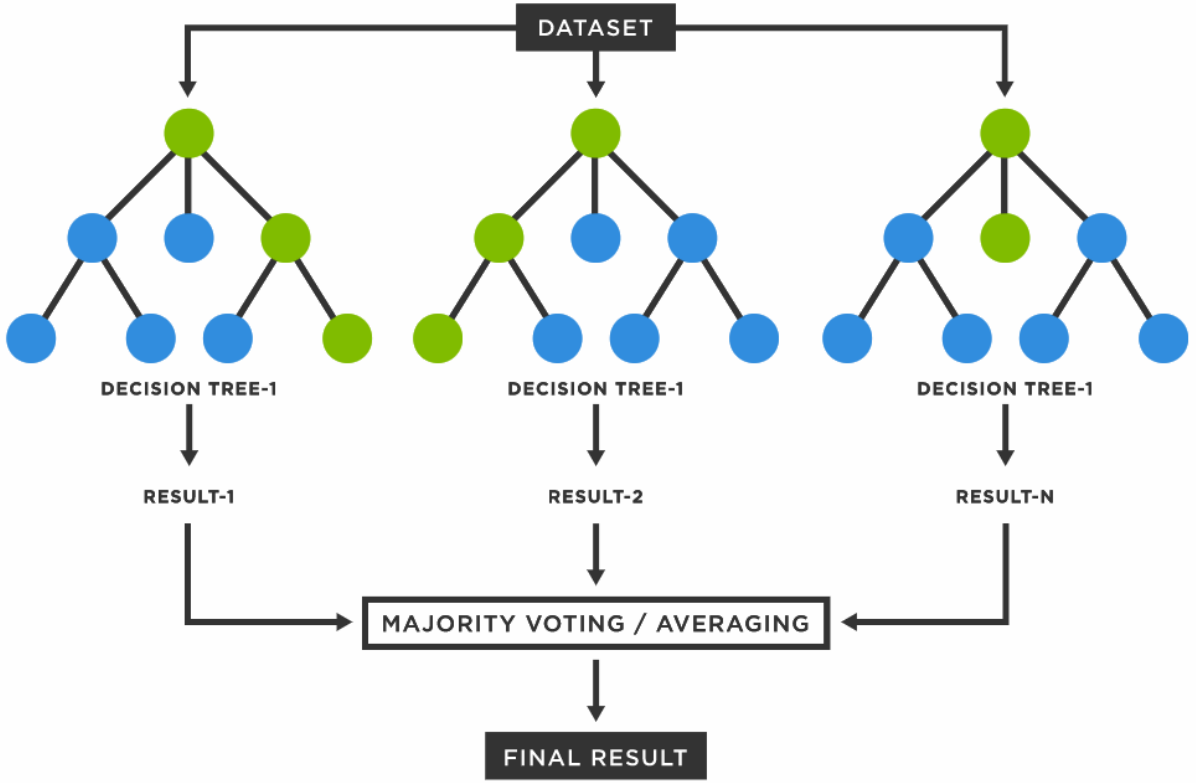

Bagging
Classifier-1
Classifier-2
Classifier-3
Parallel
Boosting
Classifier-1
Classifier-2
Classifier-3
Sequential

集成学习


single
complete training set
bagging
random sampling with replacement
boosting
random sampling with replacement over weighted data

随机森林（random forest）

- 随机森林是 Bagging 的一个扩展变体
- 随机森林是由多棵“决策树”组成的



随机森林（random forest）

- **传统的决策树在选择划分特征时，是从当前的所有 d 个特征中选择一个最优特征**

- **而当随机森林中的每棵树在选择划分特征时，先从所有 d 个特征中随机选择一个包含 k 个特征的子集，再从子集中挑选最优特征用于划分**
  - 一般推荐 $k = \log_2 d$

随机森林（random forest）

- Bagging 本身的多样性来自“随机有放回抽样”带来的样本扰动

- 随机森林的多样性出来自于样本扰动之外，还来自于“特征扰动”；因此其泛化性能可通过个体树之间的差异度进一步提升

随机森林（random forest）

- 随机森林简单、容易实现、计算开销小
- 令人惊奇的是，随机森林在很多现实任务中表现极为优异

- 随机森林中每棵数个体表现都不如传统的决策树
- 随机森林总体表现通常会收敛到更低的泛化误差

随机森林（random forest）

Prev Up Next

scikit-learn 1.1.3
Other versions

Please **cite us** if you use the software.

1.11. Ensemble methods
1.11.1. Bagging meta-estimator
1.11.2. Forests of randomized trees
1.11.3. AdaBoost
1.11.4. Gradient Tree Boosting
1.11.5. Histogram-Based Gradient Boosting
1.11.6. Voting Classifier
1.11.7. Voting Regressor
1.11.8. Stacked generalization

1.11.2. Forests of randomized trees

The `sklearn.ensemble` module includes two averaging algorithms based on randomized decision trees: the RandomForest algorithm and the Extra-Trees method. Both algorithms are perturb-and-combine techniques [B1998] specifically designed for trees. This means a diverse set of classifiers is created by introducing randomness in the classifier construction. The prediction of the ensemble is given as the averaged prediction of the individual classifiers.

As other classifiers, forest classifiers have to be fitted with two arrays: a sparse or dense array X of shape `(n_samples, n_features)` holding the training samples, and an array Y of shape `(n_samples,)` holding the target values (class labels) for the training samples:

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> X = [[0, 0], [1, 1]]
>>> Y = [0, 1]
>>> clf = RandomForestClassifier(n_estimators=10)
>>> clf = clf.fit(X, Y)
```

Like decision trees, forests of trees also extend to multi-output problems (if Y is an array of shape `(n_samples, n_outputs)`).

1.11.2.1. Random Forests

In random forests (see `RandomForestClassifier` and `RandomForestRegressor` classes), each tree in the ensemble is built from a sample drawn with replacement (i.e., a bootstrap sample) from the training set.

Furthermore, when splitting each node during the construction of a tree, the best split is found either from all input features or a random subset of size `max_features`. (See the parameter tuning guidelines for more details).

梯度提升决策树（Gradient Boosting Decision Tree）

- **梯度提升决策树（Gradient Boosting Decision Tree，GBDT）是一种基于 boosting 集成思想的决策树模型**

- **训练时采用前向分布算法进行贪婪的学习，每次迭代都学习一棵 CART 树来拟合之前 t-1 棵树的预测结果与训练样本真实值的残差**

XGBoost（eXtreme Gradient Boosting）

- **XGBoost 的基本思想和 GBDT 是类似的，但进行了许多优化**

- **XGBoost 有非常好用的工具包**
  - XGBoost Documentation — xgboost 1.7.1 documentation



特征选择
(Feature Selection)

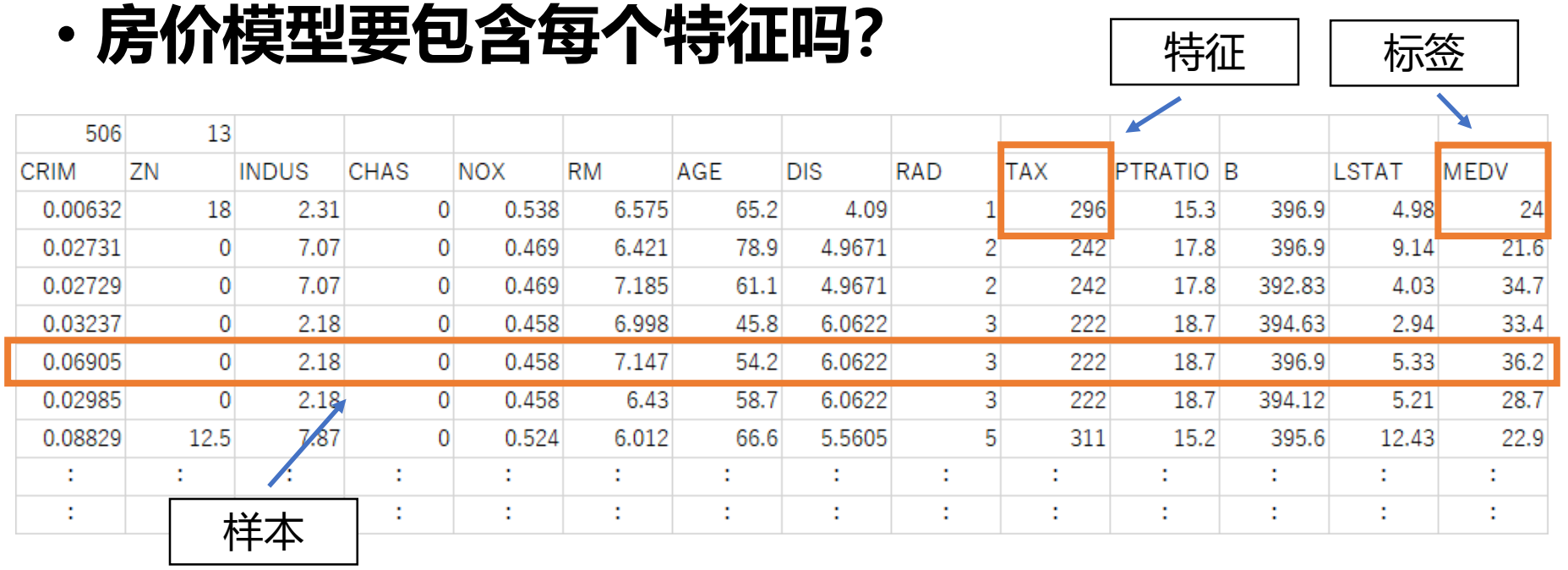
特征选择的概念

- 数据中可能有很多特征

- 对一个学习任务来说，有些特征可能很关键、很有用；另一些则可能没什么用

特征选择：波士顿房价数据

- 房价模型要包含每个特征吗？



506	13												
CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
0.02985	0	2.18	0	0.458	6.43	58.7	6.0622	3	222	18.7	394.12	5.21	28.7
0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9
:	:	:	:	:	:	:	:	:	:	:	:	:	:
:			:	:	:	:	:	:	:	:	:	:	:

属性名	解释	类型
CRIM	该镇的人均犯罪率	连续值
ZN	占地面积超过25,000平方呎的住宅用地比例	连续值
INDUS	非零售商业用地比例	连续值
CHAS	是否邻近 Charles River	离散值，1=邻近；0=不邻近
NOX	一氧化氮浓度	连续值
RM	每栋房屋的平均客房数	连续值
AGE	1940年之前建成的自用单位比例	连续值

特征与标签的说明

DIS	到波士顿5个就业中心的加权距离	连续值
RAD	到径向公路的可达性指数	连续值
TAX	全值财产税率	连续值
PTRATIO	学生与教师的比例	连续值
B	1000(BK - 0.63)^2，其中BK为黑人占比	连续值
LSTAT	低收入人群占比	连续值
MEDV	同类房屋价格的中位数	连续值

特征选择的概念

- **从给定的特征集合中选择出相关特征子集的过程，称为“特征选择”**

- **为什么要进行特征选择**
  - 降低维度，简化计算
  - 留下最关键的因素，便于解释

特征选择的关键环节

- 子集搜索：假设所有特征组成集合 $\{a_1, a_2, ..., a_d\}$
  - 前向（forward）搜索：从空集开始，逐一加入最“有用”的特征
  - 后向（backward）搜索：从全集开始，逐一去掉最“没用”的特征

一般来说，前向搜索和后向搜索都是“贪心的”。

特征选择的关键环节

- 子集评价：对于特征集合 $\{a_1, a_2, ..., a_d\}$ 的一个子集 A，如何评价它的好坏？

  思路：比较基于特征子集 A 对样本进行的划分，和真实标签对样本信息的真实划分

常见的特征选择方法类别

- 将特征子集搜索机制，与特征子集评价机制相结合，即可得到特征选择方法

- 常见的特征选择方法大致可分为三类：过滤式（filter）、包裹式（wrapper）和嵌入式（embedding）

常见的特征选择方法类别

- **过滤式（filter）：先对初始特征进行“过滤”，得到特征选择结果后，再训练学习器（特征选择与后续学习器无关）**
  - 比如，过滤可以基于相关统计量，去除冗余的特征



常见的特征选择方法类别

- 包裹式（wrapper）：特征选择与学习器训练结合，为给定的机器学习模型“量身定做”特征子集



常见的特征选择方法类别

- **嵌入式（embedding）：在机器学习模型训练过程中自动进行特征选择**
  - 例子：L1 正则化（Lasso）



常见的特征选择方法类别

- **嵌入式特征选择例子：L1 正则化（Lasso）**

$$(w^*, b^*) = \arg\min_{(w,b)} \left[\sum_{i=1}^{m} (y_i - \boldsymbol{w}^T \boldsymbol{x}_i - b)^2 + \lambda \sum_{j}^{d} |w_j| \right]$$

Lasso正则化的“特征选择”功能：对多元线性回归，加入Lasso正则化项之后，会有更多的 w_i 项为零

Python 机器学习开发环境

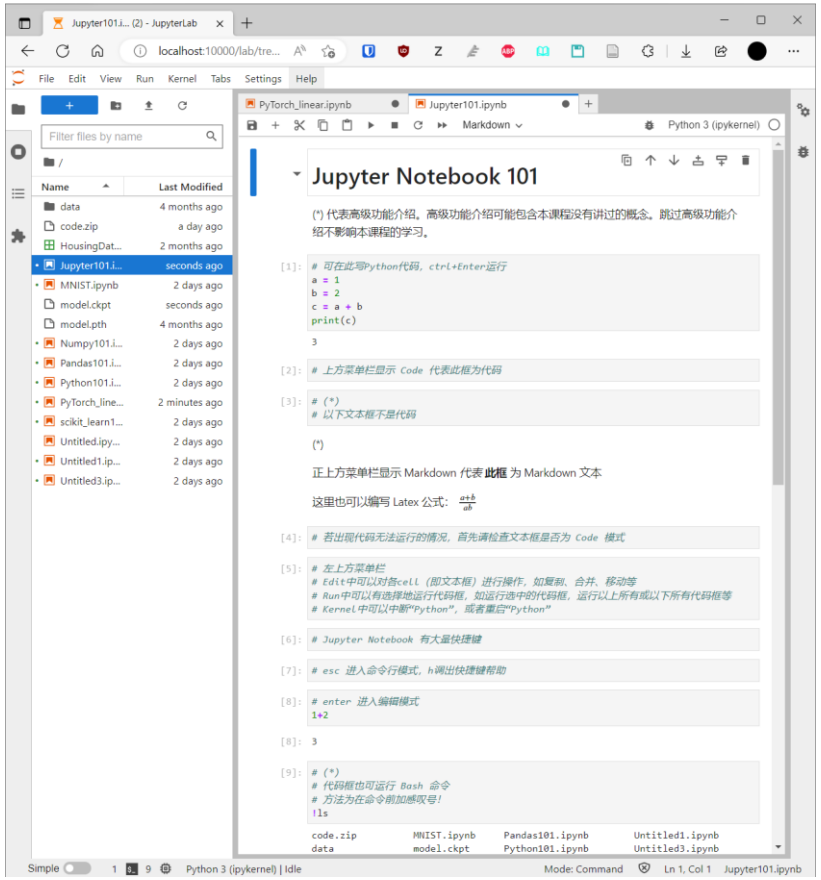
Python **开发环境**

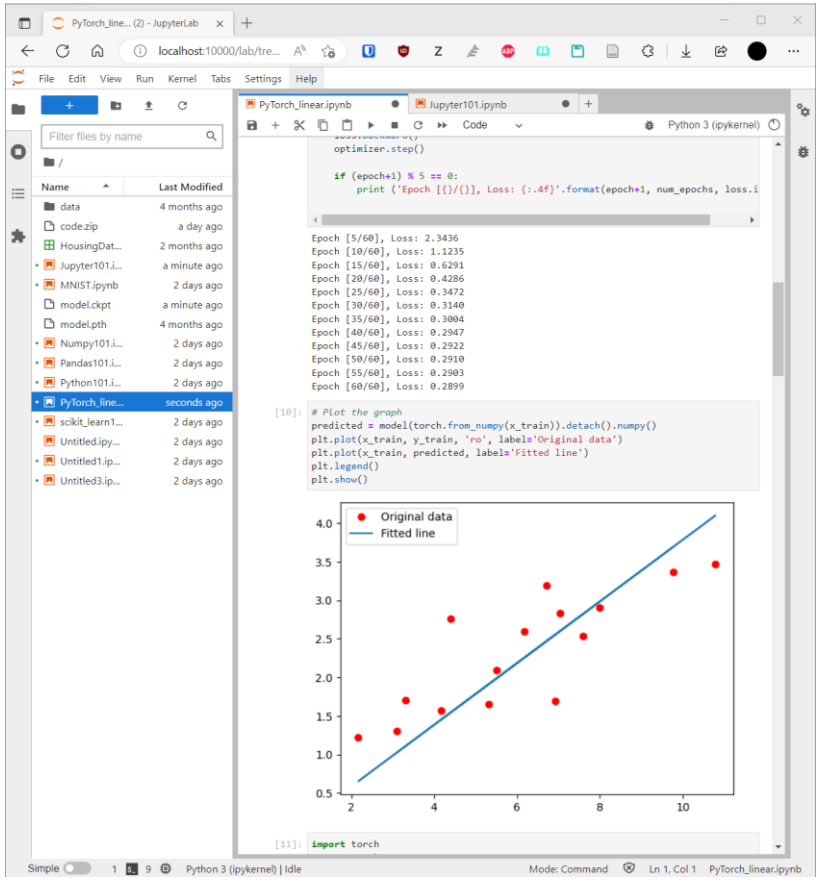
- **本课程使用 Jupyter Notebook 作为主要的编程工具**

- **本课程介绍两种搭建 Python 机器学习开发环境**（安装 Python，安装 Jupyter Notebook，安装 scikit-learn、PyTorch等工具包）**的方法**
  - 基于 Anaconda （工程实践中心机房使用的方法）
  - 基于 Docker （推荐自行安装时的方法）

Jupyter Notebook

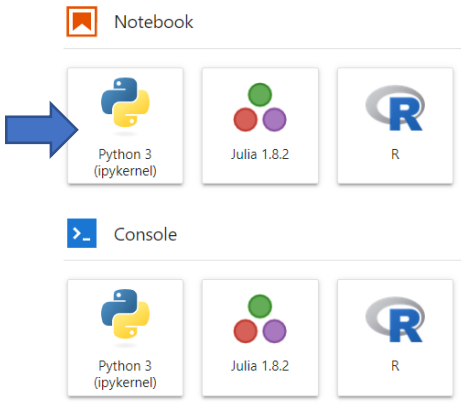
- **Jupyter Notebook是基于网页的用于交互计算的应用程序**

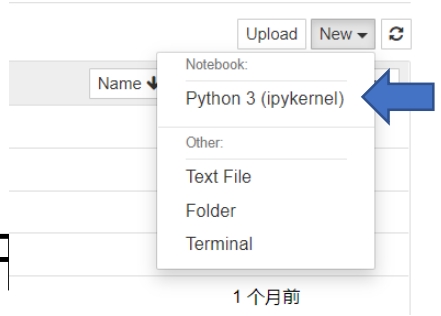




Jupyter Notebook

- **本课程使用 Jupyter Notebook 中的 Python 3 (ipykernel) Notebook**
  - 新建：不同版本新建 Python 3 (ipykernel) Notebook 的方式略有不同





  - 打开：也可以 ook 直接打开 （如本课程提供的上机

Anaconda

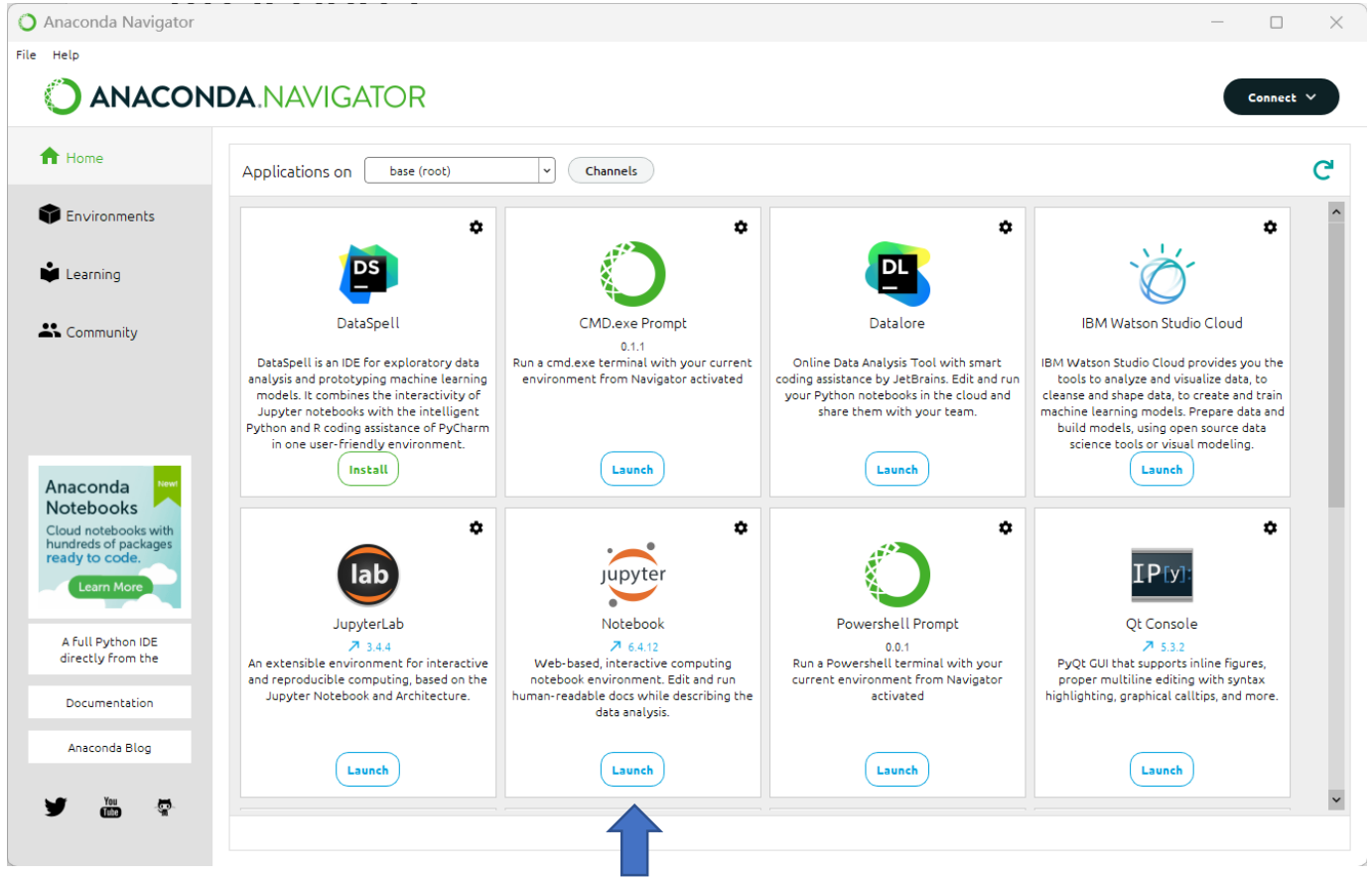
- **Anaconda，是一个开源的Python发行版本，其包含了180多个科学包及其依赖项**

- **Anaconda 包含了 Jupyter Notebook，scikit-learn，但没有包含 PyTorch（神经网络工具包）**
  - 如要使用 PyTorch，需手动安装
    - conda install pytorch
    - conda install torchvision

- **机房的 Anaconda 已经配置好（已装好 PyTorch）**

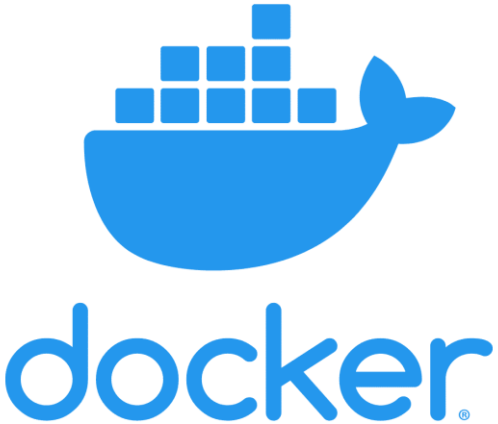
Anaconda

- **下载安装好 Anaconda 后，启动 Anaconda Navigator，然后点击 Jupyter Notebook Launch**



Docker

- **Docker 是一个开源的应用容器引擎，让开发者可以打包他们的应用以及依赖包到一个可移植的镜像中，然后发布到任何流行的Linux或Windows操作系统的机器上，也可以实现虚拟化**



Docker **应用的背景**

- **代码版本兼容问题**
  - Python 等程序的代码依赖许多个独立发布的工具包，每个工具包都有可能在不同的时候升级版本
    - 任何工具包版本变化都有可能会导致代码无法运行
  - 如何把代码和其所需的所有工具包（即“环境”）打包？
    - 一台计算机，如何可以同时运行多个环境和对应的代码；同时，又不像多个虚拟机那样消耗大量资源
- **为解决上述问题，Docker技术由此产生**

Docker是什么

- **Docker简介**

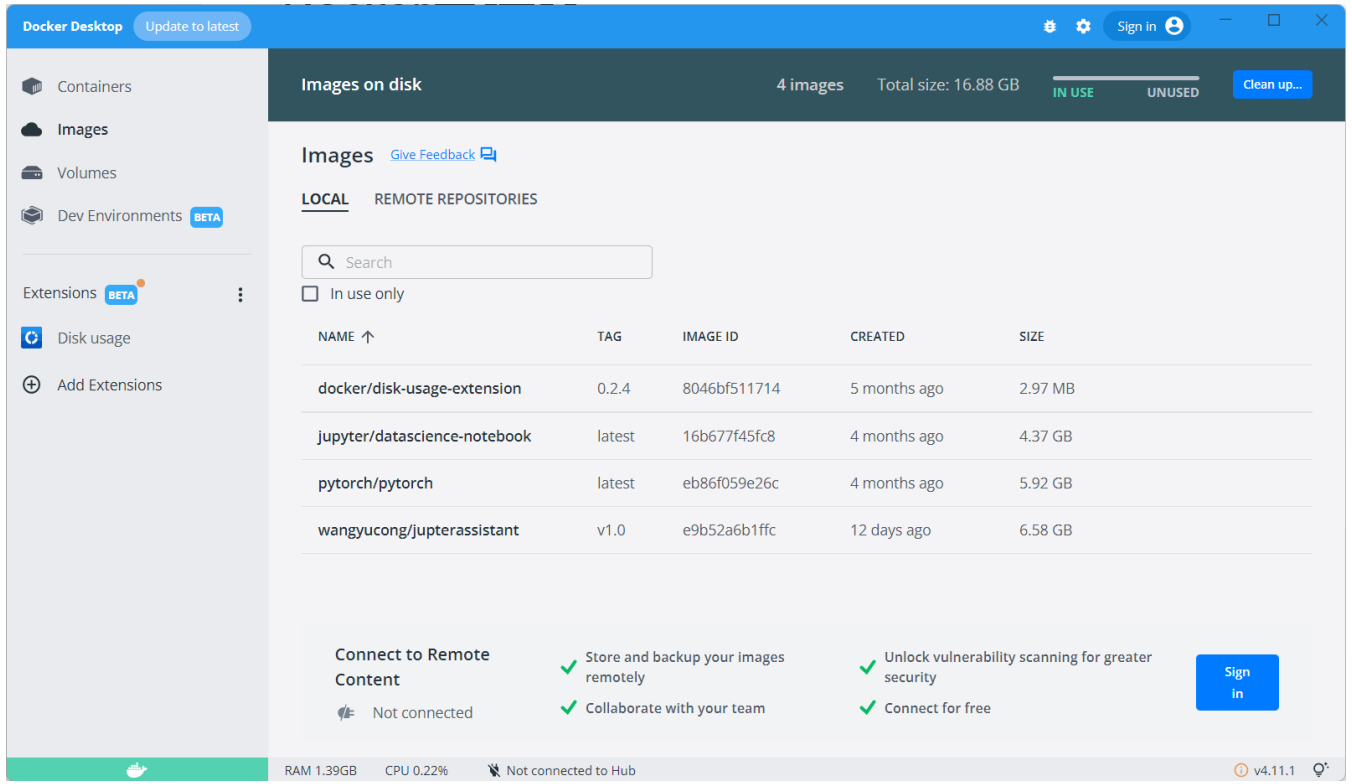
Docker是一个开源的引擎，可以轻松的为任何应用创建一个轻量级的、可移植的、自给自足的容器。开发者在笔记本上编译测试通过的容器可以批量地在生产环境中部署，包括VMs（虚拟机）、 bare metal、OpenStack 集群和其他的基础应用平台。

Docker通常用于如下场景:
1.web应用的自动化打包和发布;
2.自动化测试和持续集成、发布;
3.在服务型环境中部署和调整数据库或其他的后台应用;
4.从头编译或者扩展现有的OpenShift或Cloud Foundry平台来搭建自己的PaaS环境。

Docker在windows系统下安装方法:
https://www.runoob.com/docker/windows-docker-install.html
（在自己的电脑上安装方法，机房电脑不需要安装）

Docker使用

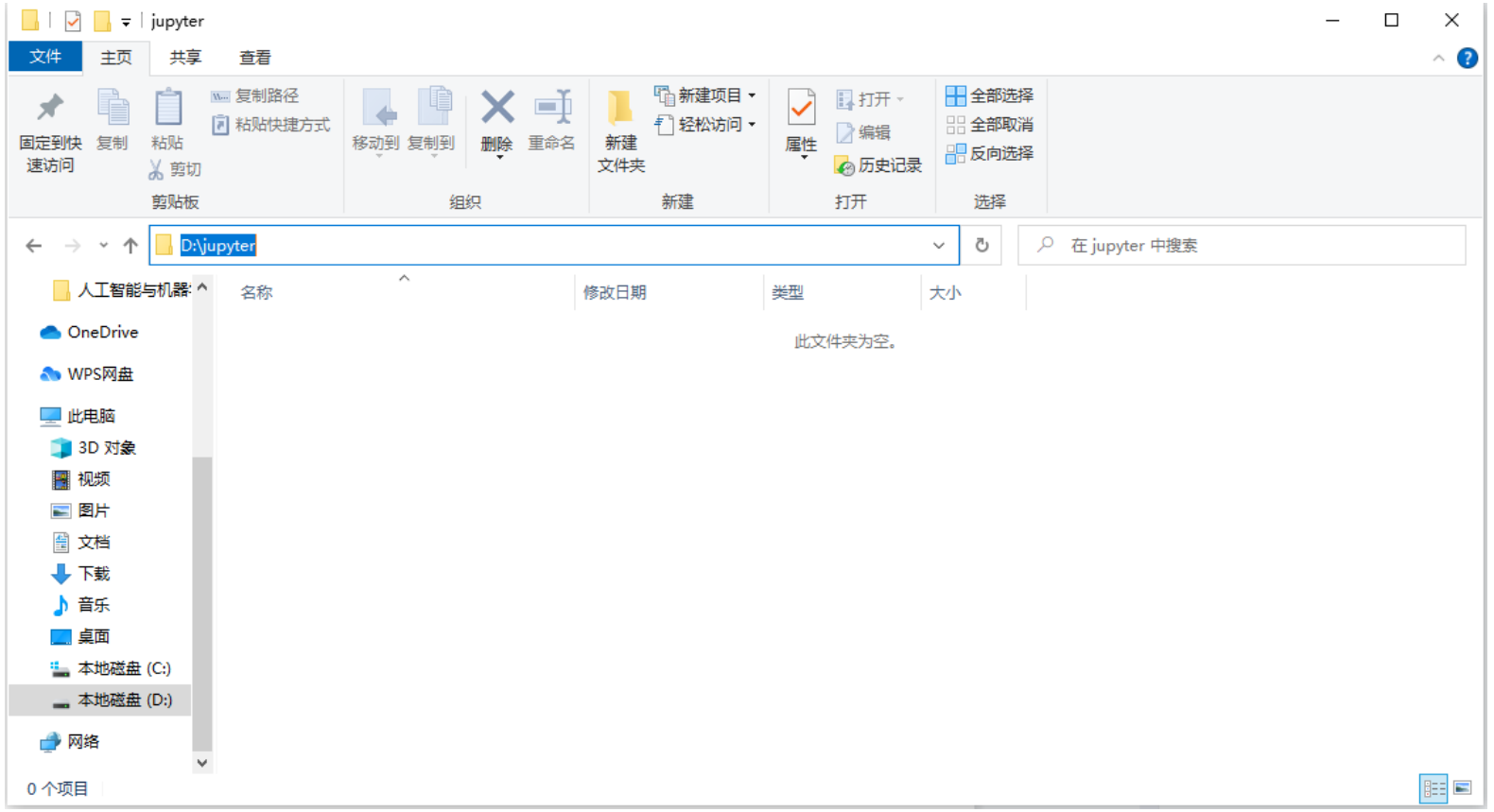
- **安装成功后，先运行 Docker Desktop**



- **余下的工作需在命令行完成**

Docker使用

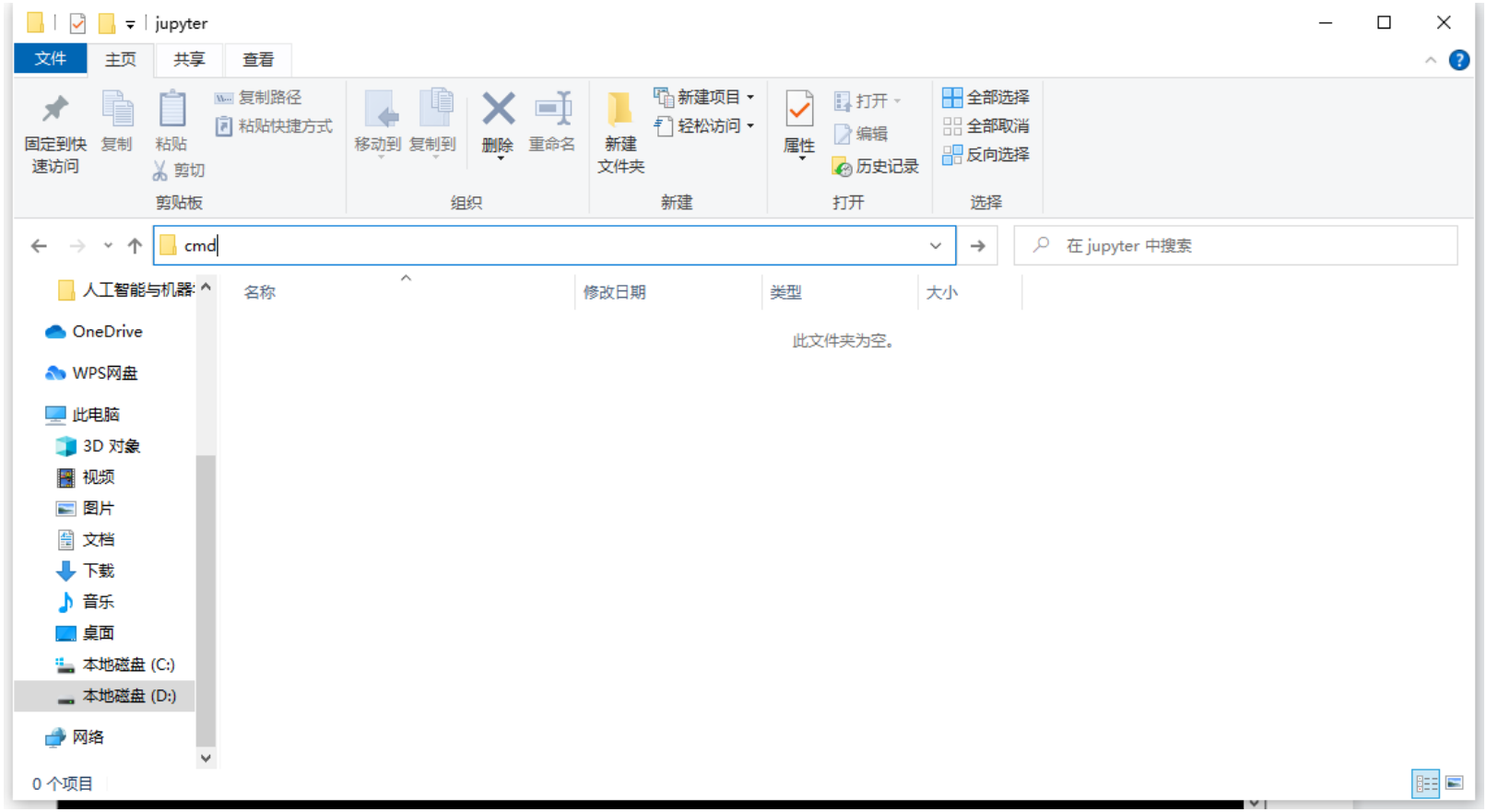
- **下载本课程所需的镜像**



打开文件夹（文件夹中任一目录都可以，这里以D:\jupyte为例）

Docker使用

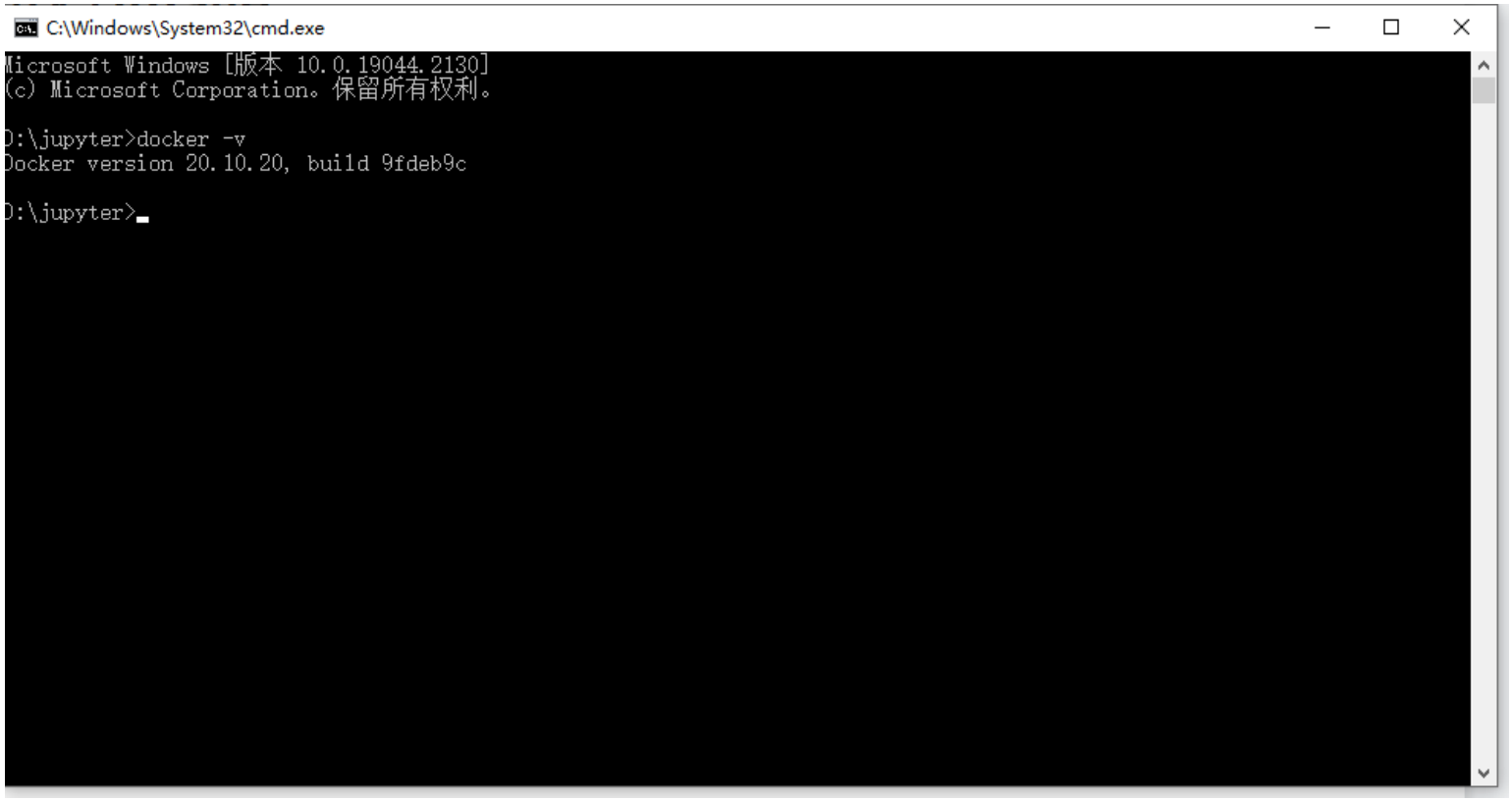
- **下载本课程所需的镜像**



输入cmd，按下enter，进入cmd界面

Docker使用

- **下载本课程所需的镜像**

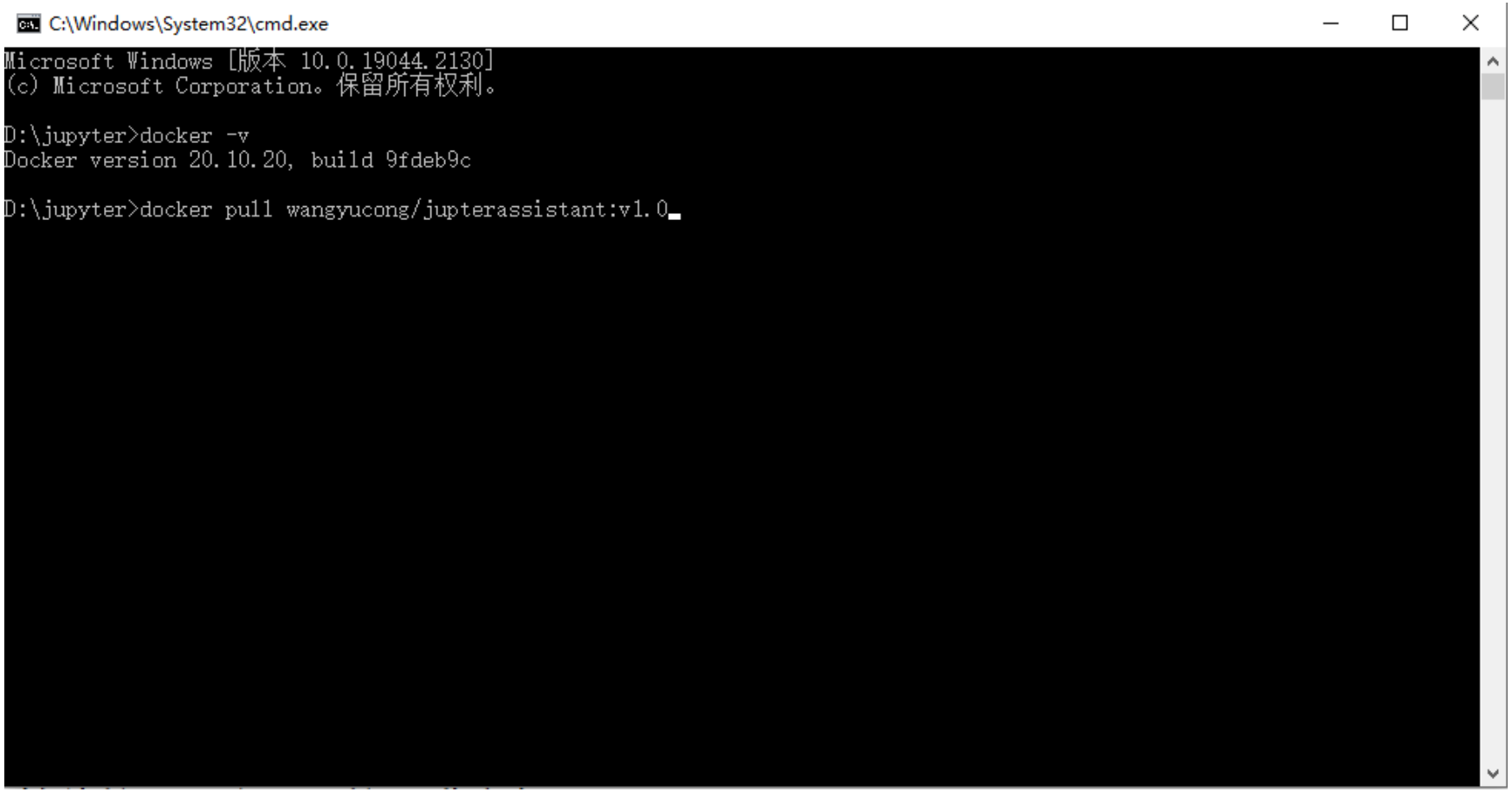


输入docker -v，返回docker版本则说明docker安装没问题

Docker使用

- **下载本课程所需的镜像**

输入docker pull wangyucong/jupterassistant:v1.0

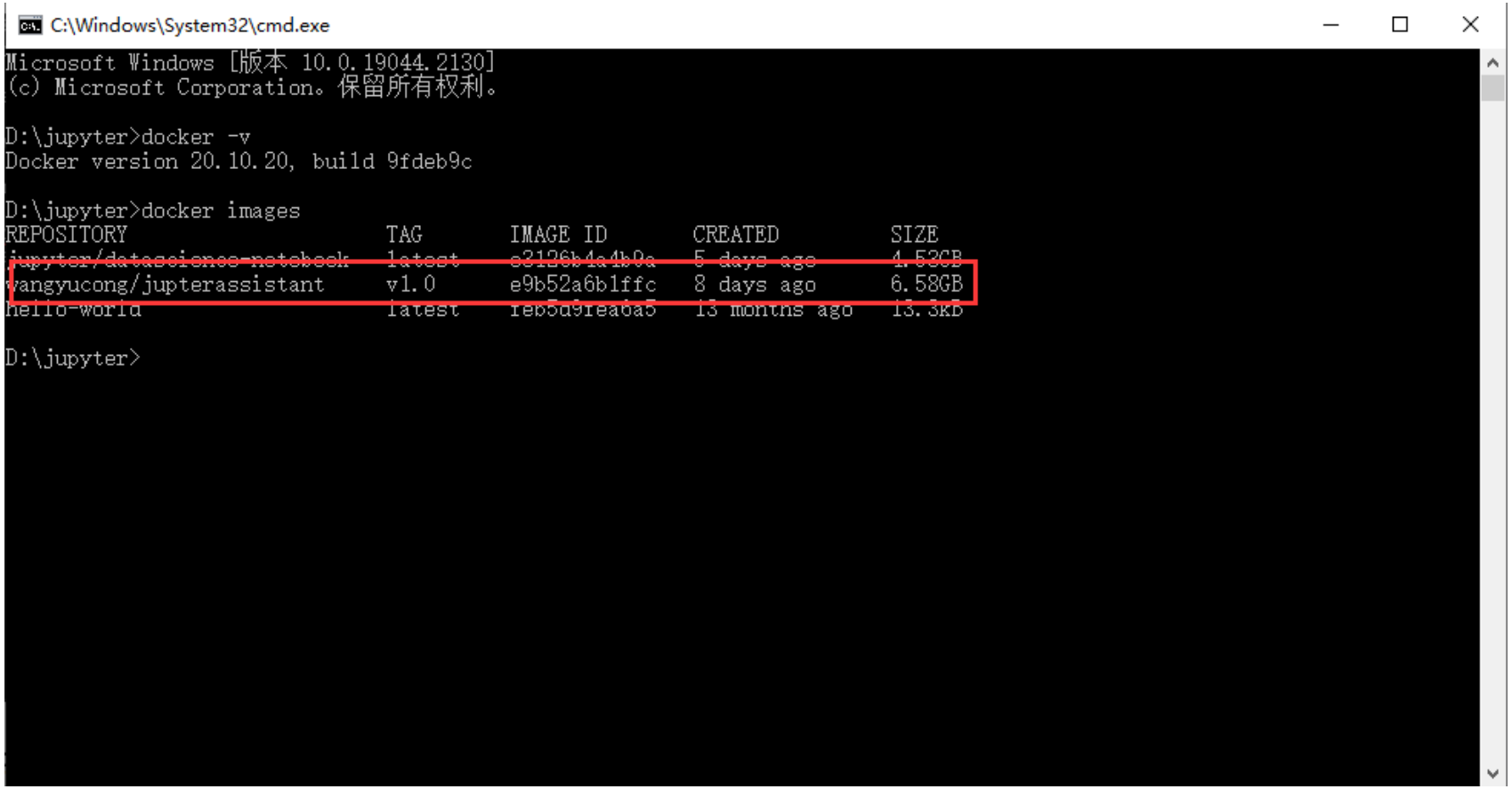


等待拉取，会显示拉取进度条

Docker使用

- **下载本课程所需的镜像**

输入docker images指令



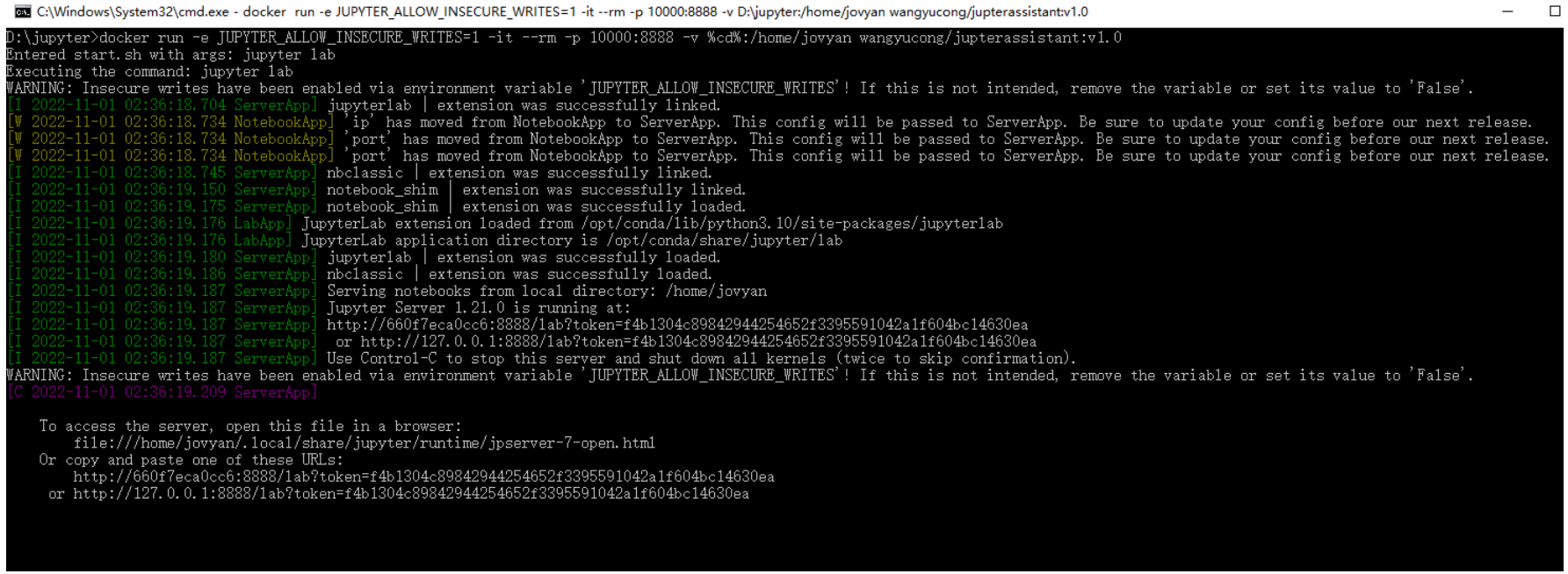
如红色框所示，拉取镜像成功

Docker使用

- **运行镜像**

然后在cmd中输入（Windows系统）

docker run -e JUPYTER_ALLOW_INSECURE_WRITES=1 -it --rm -p 10000:8888 -v %cd%:/home/jovyan wangyucong/jupterassistant:v1.0



出现图中信息则代表运行成功

Docker使用

- **运行镜像**

Ubuntu系统下需要输入

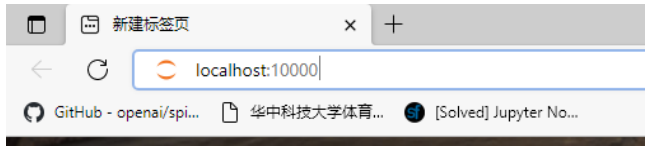
sudo docker run -e JUPYTER_ALLOW_INSECURE_WRITES=1 -it --rm -p 10000:8888 -v $(pwd):/home/jovyan wangyucong/jupterassistant:v1.0

出现图中信息则代表运行成功

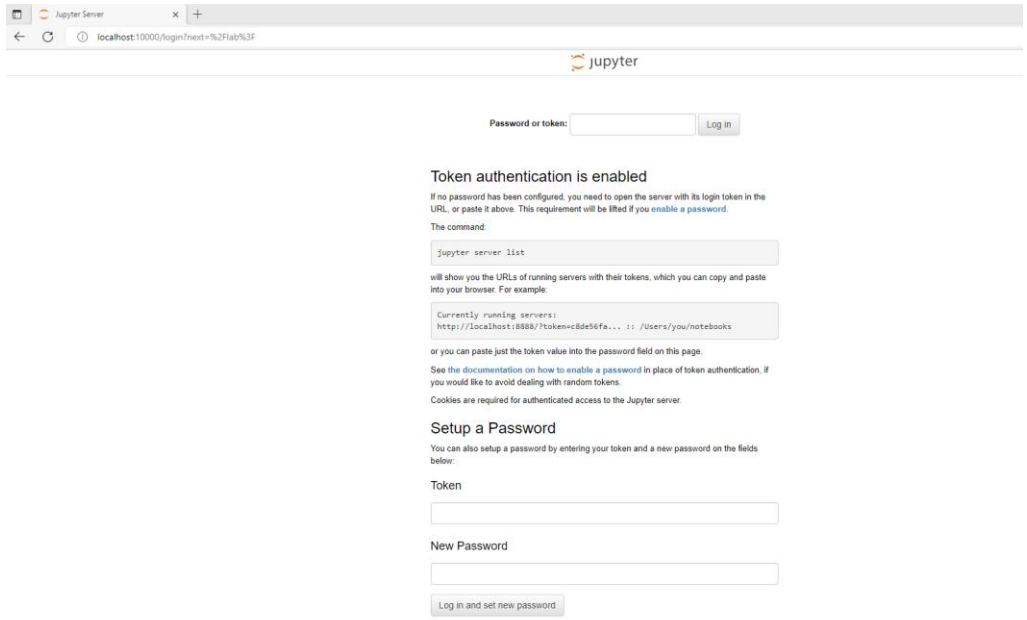
Docker使用

- **连接 Jupyter Notebook**

打开任意浏览器，在浏览器地址栏中输入 localhost:10000



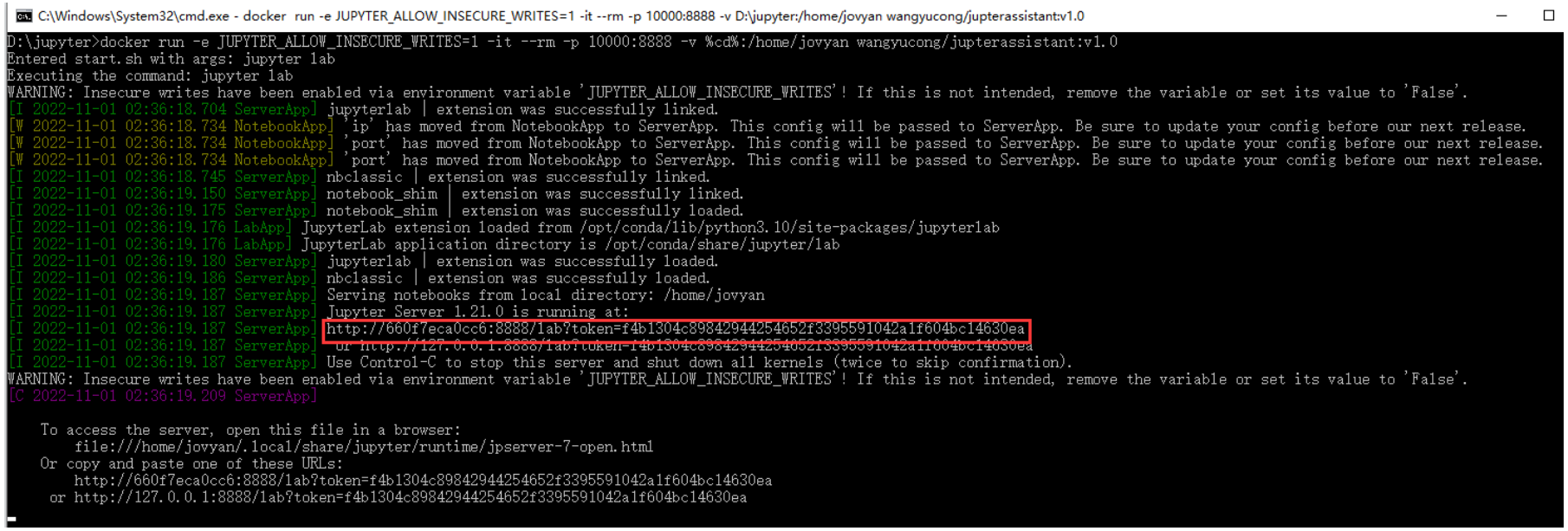
出现下图中信息则代表运行jupyter成功



Docker使用

- **连接 Jupyter Notebook**

在Password or token中复制cmd窗口中的密钥。如下图红色框中所示

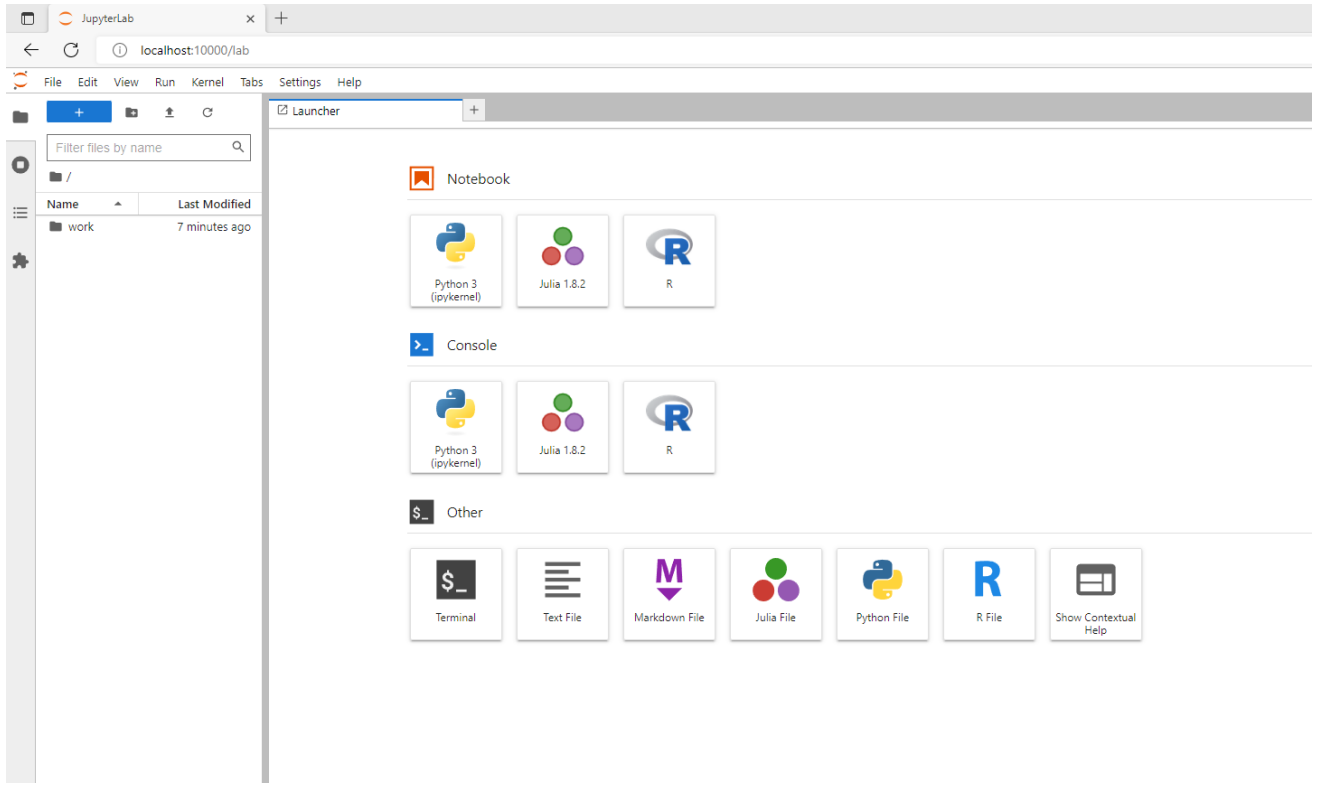


注：复制“token=”后面的密钥

Docker使用

- **连接 Jupyter Notebook**

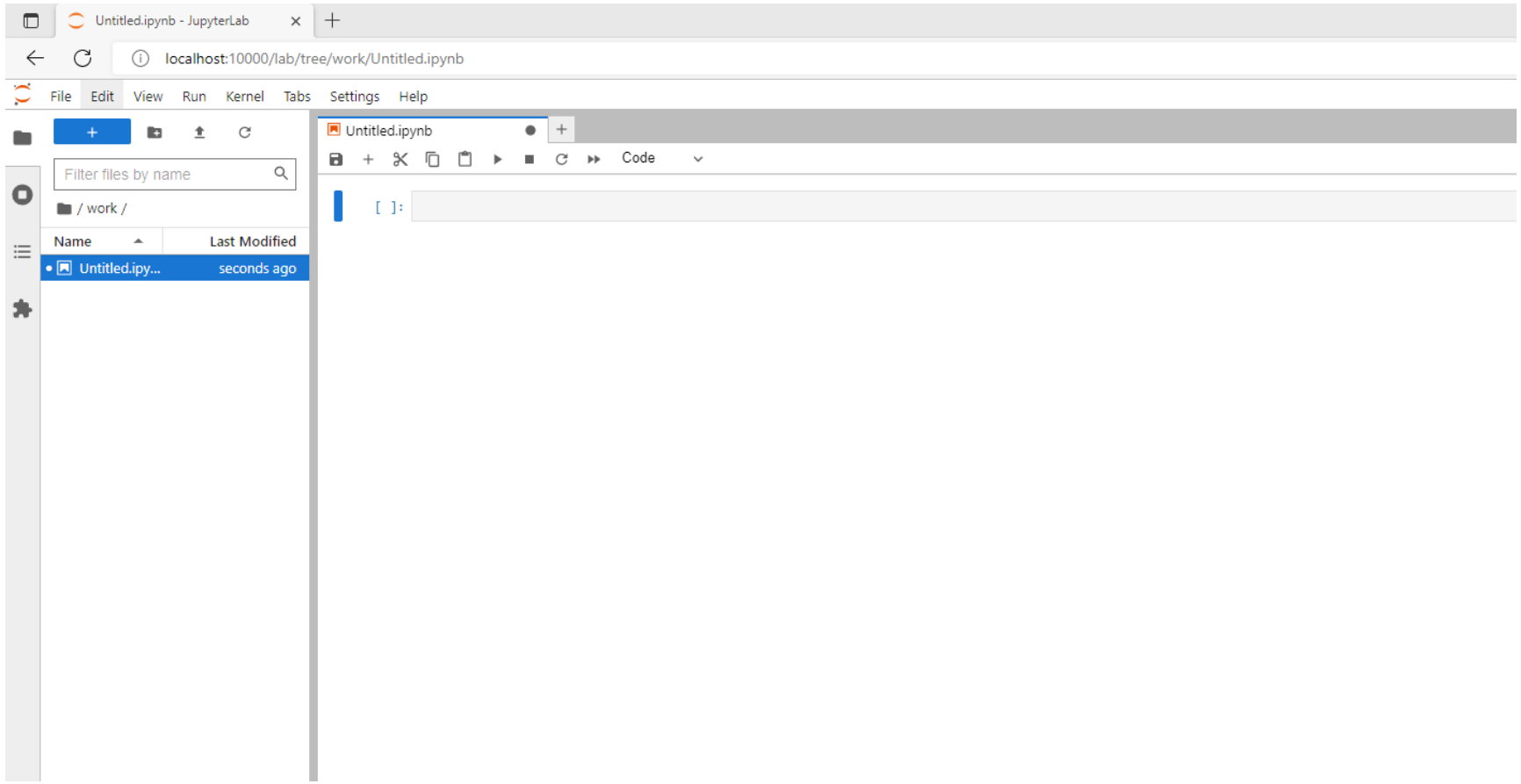
点击Log in后界面如下所示



Docker使用

- **连接 Jupyter Notebook**

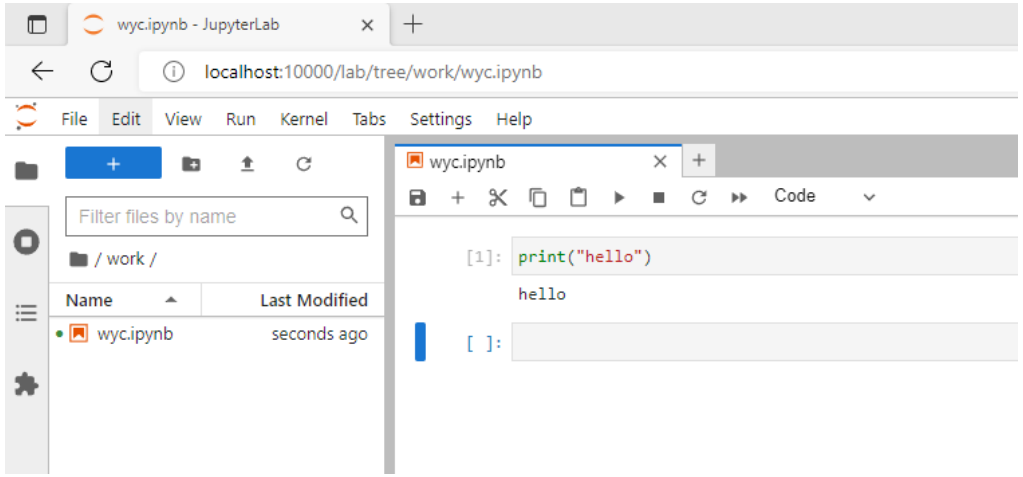
点击notebook中的python3，进入python编辑界面

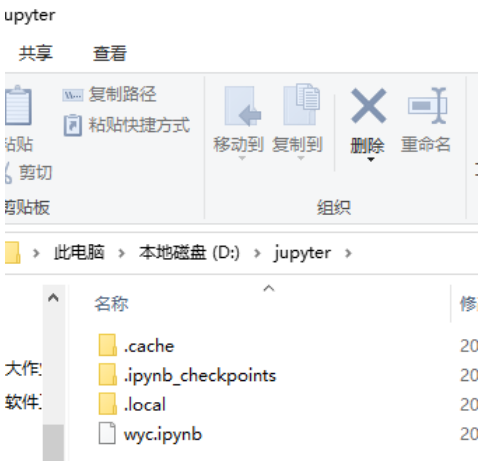


Docker使用

- **连接 Jupyter Notebook**

运行python代码后ctrl+s保存代码





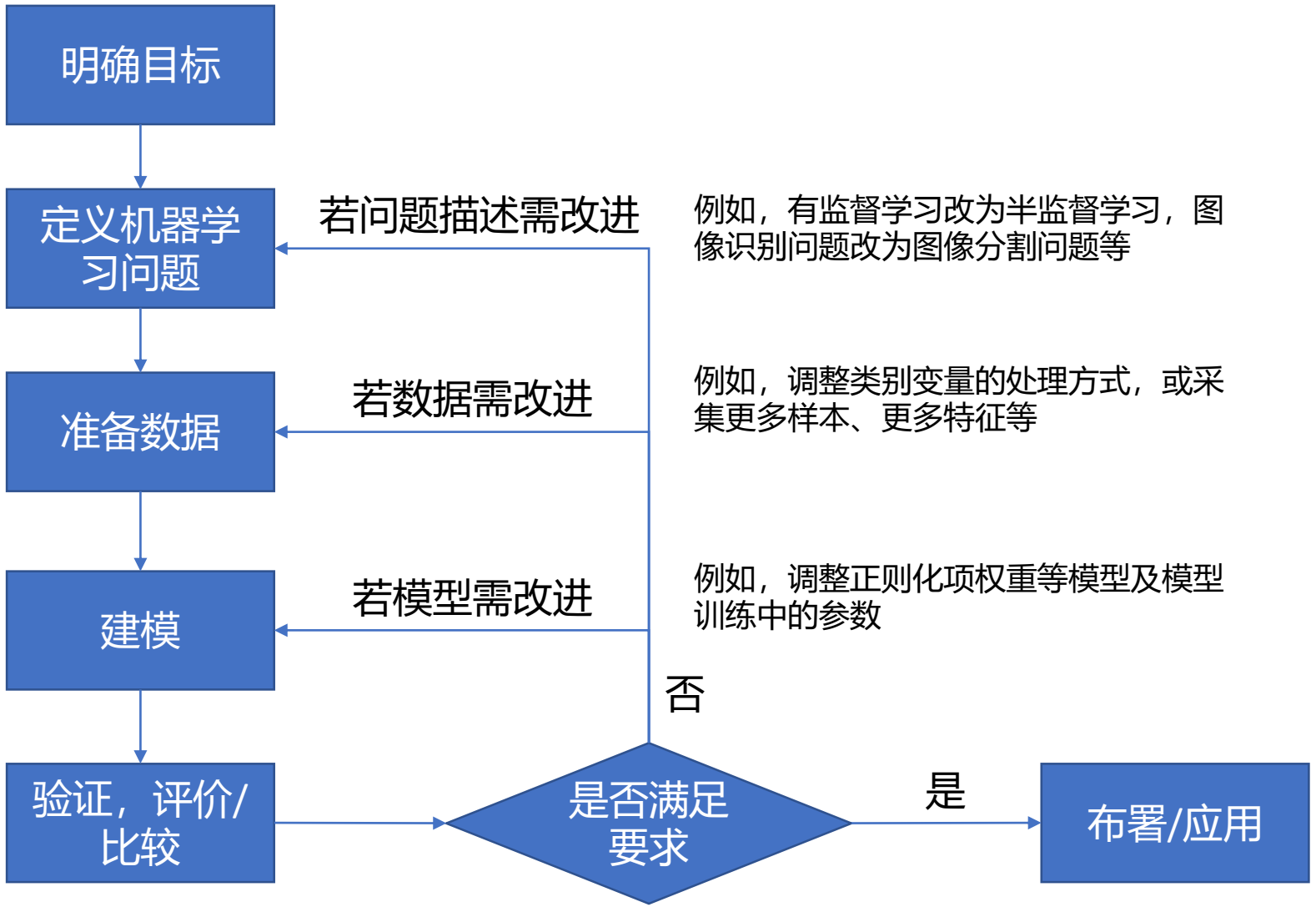
同时可以在本地文件夹中找到保存的代码文件

机器学习应用的开发流程

开发流程

- 明确项目的大目标
- 定义机器学习问题
- **准备数据**（可能需要大量时间）
- **建模**，验证评价，**迭代改进**（可能需要大量时间）
- 布署/应用

开发流程


明确目标
定义机器学习问题
准备数据
建模
验证，评价/比较
是否满足要求
布署/应用
若问题描述需改进
若数据需改进
若模型需改进
否
是
例如，有监督学习改为半监督学习，图像识别问题改为图像分割问题等
例如，调整类别变量的处理方式，或采集更多样本、更多特征等
例如，调整正则化项权重等模型及模型训练中的参数

开发流程：明确项目的大目标

- **使用机器学习是为了解决什么问题（不仅仅是机器学习模型的目标，而是项目的大目标）**

- **例如**
  - 为产品实现一个xxxxxx的新功能
  - 提前预测客户的行为（如商品订单），目的是提前做好准备（如提前将商品发往相应仓库），以缩短物流时间

开发流程：定义机器学习问题

- **本机器学习的目标是什么?**

- **确定机器学习模型的输出（即有监督情况下的标签）**
  - 例如：识别机器人周边的行人，预测未来的轨迹

- **确定机器学习类型：有监督/无监督/半监督/主动**

开发流程：数据准备

- **是已经有了数据，还是需要新采集**
  - 是否有开源数据可供使用
  - 现有数据能否支撑本机器学习
    - 数据质量：数据可靠程度，测量误差与噪音，缺失值情况等
    - 数据的多样性与代表性，数据的分布与目标的分布是否一致
- **（对有监督学习）如何获得数据标签**

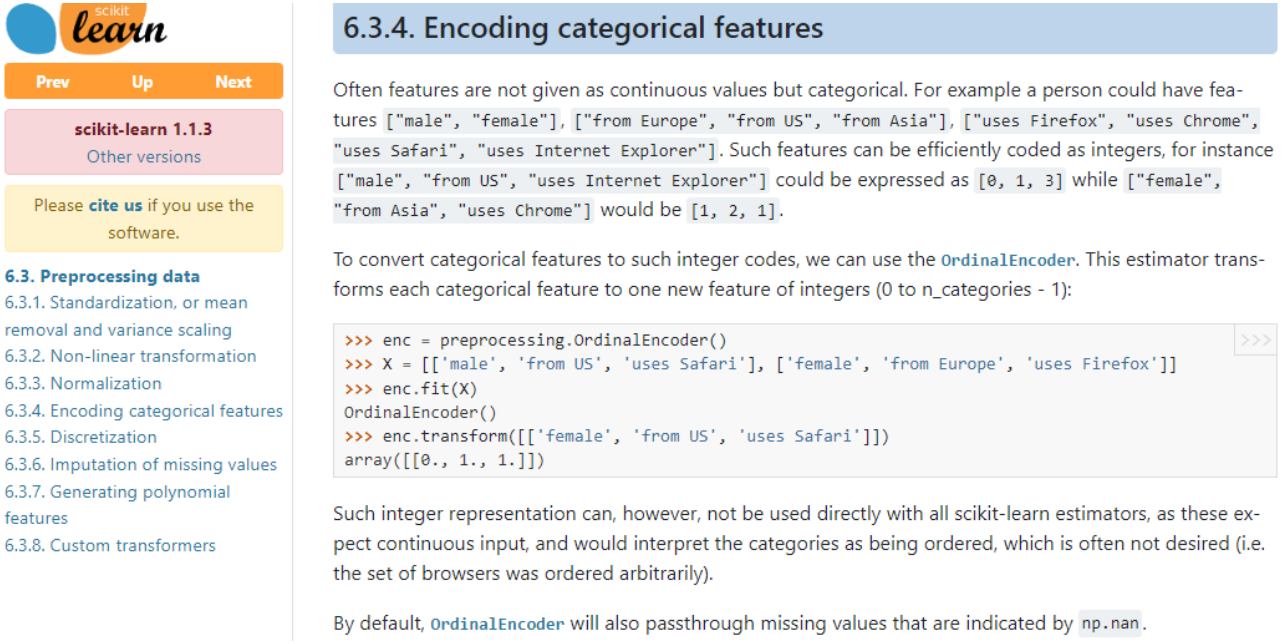
- **数据处理：将原始数据转化成可用于机器学习的数据**

数据准备：归一化（Normalization）

- **有些机器学习数值算法对数据的分布比较敏感**
  - 对于样本(0.1,10000)这种特征数值差异很大的情况，数值计算结果往往效果不佳
- **可以考虑进行归一化，即对所有样本的同一个特征进行线性变化，使各特征数值上相差不太大**
  - 线性归一 $$x_n = \frac{x - \min(x)}{\max(x) - \min(x)}$$
  - 标准差归一 $$x_n = \frac{x - \mu}{\sigma}$$

数据准备：类别特征（categorical features）处理

- **类别特征**
  - 序列（ordinal）编码
    - 学士：1，硕士：2，博士：3
  - 独热（one-hot）编码
    - 狗：(1,0,0)；猫：(0,1,0)；兔：(0,0,1)



scikit learn

Prev Up Next

scikit-learn 1.1.3
Other versions

Please cite us if you use the software.

6.3. Preprocessing data
6.3.1. Standardization, or mean removal and variance scaling
6.3.2. Non-linear transformation
6.3.3. Normalization
6.3.4. Encoding categorical features
6.3.5. Discretization
6.3.6. Imputation of missing values
6.3.7. Generating polynomial features
6.3.8. Custom transformers

6.3.4. Encoding categorical features

Often features are not given as continuous values but categorical. For example a person could have features `["male", "female"]`, `["from Europe", "from US", "from Asia"]`, `["uses Firefox", "uses Chrome", "uses Safari", "uses Internet Explorer"]`. Such features can be efficiently coded as integers, for instance `["male", "from US", "uses Internet Explorer"]` could be expressed as `[0, 1, 3]` while `["female", "from Asia", "uses Chrome"]` would be `[1, 2, 1]`.

To convert categorical features to such integer codes, we can use the `OrdinalEncoder`. This estimator transforms each categorical feature to one new feature of integers (0 to n_categories - 1):

```
>>> enc = preprocessing.OrdinalEncoder()
>>> X = [['male', 'from US', 'uses Safari'], ['female', 'from Europe', 'uses Firefox']]
>>> enc.fit(X)
OrdinalEncoder()
>>> enc.transform([['female', 'from US', 'uses Safari']])
array([[0., 1., 1.]])
```

Such integer representation can, however, not be used directly with all scikit-learn estimators, as these expect continuous input, and would interpret the categories as being ordered, which is often not desired (i.e. the set of browsers was ordered arbitrarily).

By default, `OrdinalEncoder` will also passthrough missing values that are indicated by `np.nan`.

数据准备：时间特征处理

- **例：2022-11-08 17:00:00.0000**
- **根据问题的具体特点，分别提取年、月、日、小时、甚至星期等作为特征**
  - 例如，预测每天商店顾客流量：可考虑将否周末、是否节假日作为特征（星期几远比几号更重要）；
  - 提醒：有必要的话，考虑时差的影响（即使用当地时间）

开发流程：建模，验证评价，迭代改进

- **确定评价指标**
  - 评价指标与项目大目标相关
  - 指标可能是多元的

- **迭代改进**
  - 根据具体情况，改进模型中的预设参数、模型的选择，改进数据，改进机器学习问题的设定等

开发流程：布署/应用

- 离线分析 vs 在线应用
- 如何将训练好的模型提供给下游应用?
- 布署后模型是否需要改进?
- 布署后是否要收集新的数据?

案例：汽车故障预诊断

数据说明

- **汽车VIN**
  - Vehicle Identification Number (车辆识别码)
  - 每辆车，都有一个17位的唯一识别码



数据一

- 汽车车型、配置与零部件

VIN	车型
XXXXXXXXXXXXXXXX1	55K尊贵型
XXXXXXXXXXXXXXXX2	55K豪华型

车型	55KM舒适型	55KM豪华型	55KM尊贵型
建议零售价(元)	124,600	134,600	140,600
综合补贴后价格（元）	119,800	129,800	135,800
BSD盲区监测系统	—	—	•
RCTA后方交通穿行提示	—	—	•
FCW前向碰撞预警系统	—	—	•
RCW后向碰撞预警系统	—	—	•
LCA并线辅助系统	—	—	•

- + 每个VIN对应的每个零部件识别码或批次代码
- + 每个零部件识别码或批次对应的供应商信息

数据二

- 总装线生产工序与状态

VIN	工序号	工序状态	工序开始时间
XXXXXXXXXXXXXXXX3	1	正常	12:12:12
XXXXXXXXXXXXXXXX4	1	异常A	12:12:32
XXXXXXXXXXXXXXXX3	2	正常	12:12:32
XXXXXXXXXXXXXXXX4	2	异常B	12:12:52

数据三

- 下线检测数据

VIN	检测项目号	检测结果	返修处理代码
XXXXXXXXXXXXXXXX3	1	通过	
XXXXXXXXXXXXXXXX4	1	通过	
XXXXXXXXXXXXXXXX3	2	通过	
XXXXXXXXXXXXXXXX4	2	不通过	A05
……			
XXXXXXXXXXXXXXXX4	2	不通过	A06
……			
XXXXXXXXXXXXXXXX4	2	通过	

数据说明

- **故障诊断码DTC（Diagnostic trouble code）**
  - 车辆在运行中，系统自检后生成的
  - 产生故障诊断码，并不一定代表车出现故障需要修理
    - 会有（大量）误判
    - 会有不影响功能的“故障”
    - 假设：相当一部分车，在运行时，都会产生几个DTC

DTC	说明	DTC 类型
B1401	远程信息-ECU CPU 通信和 EEPROM 错误	内部错误
B1403	远程信息-ECU GPS 接收器错误	内部错误
B1404	远程信息-ECU 电话单元错误	内部错误
B1412	TEL 天线断开	信号错误
B1413	TEL 天线对搭铁短路	信号错误
B1414	GPS 天线断开	信号错误
B1415	GPS 天线对搭铁短路	信号错误
B1416	TELE 控制台开关总成断开	信号错误
B1417	TELE 控制台开关对搭铁短路	信号错误
B1425	CDS（碰撞检测信号）输入电路故障	信号错误
B1775	麦克风输入/输出断路或对电源短路	信号错误
B1776	麦克风输入/输出对搭铁短路	信号错误
U1280	通信总线线路故障（远程信息 ECU 检测出总线关闭状态）	通信丢失
U128D	与 B-CAN 上的仪表失去通信	通信丢失
U1281	与 MICU 失去通信（MICU 框架）	通信丢失

牛车宝 www.ncboo.com

数据四

- 运行中的故障诊断码数据

VIN	DTC	产生时间
XXXXXXXXXXXXXXXX3	P0505	2022-05-05 12:12:12
XXXXXXXXXXXXXXXX3	P0507	2022-05-05 12:12:12
XXXXXXXXXXXXXXXX3	B0105	2022-05-05 12:12:42
XXXXXXXXXXXXXXXX3	P0505	2022-05-06 12:12:12

数据五

- 汽车运行的启动、停止数据

VIN	启动时间	停止时间
XXXXXXXXXXXXXXXX5	2022-05-05 12:12:12	2022-05-05 12:12:16
XXXXXXXXXXXXXXXX5	2022-05-05 12:12:17	2022-05-05 12:14:12
XXXXXXXXXXXXXXXX5	2022-05-06 12:10:00	2022-05-06 15:00:42
XXXXXXXXXXXXXXXX5	2022-05-06 15:10:44	2022-05-06 15:42:00

数据六

- **客服通话数据**
  - VIN + 电话号码 + 时间 + 录音

数据七

- **道路救援数据**
  - VIN + 电话号码 + 时间 + 救援原因
  - 救援原因
    - 事故
    - 无法开门
    - 没油
    - 无法启动车辆
    - 可以启动车辆，但无法前进
    - 刹车故障
    - ……

数据八

- **4S店维修数据**
  - VIN
  - 故障描述文字
  - 检测项目代码，检测项目结果
  - 维修项目代码
  - 零件更换记录

数据总结

- 数据一：汽车车型、配置与零部件
- 数据二：总装线生产工序与状态（异常数据极少）
- 数据三：下线检测数据（异常数据较少）
- 数据四：运行中的故障诊断码数据（数据很多）
- 数据五：汽车运行的启动、停止数据（数据极多）
- 数据六：客服通话（数据较少）
- 数据七：道路救援（数据极少）
- 数据八：维修数据（数据较少）

讨论

- **假如希望开发故障预诊断功能，将来通过OTA更新**

  - 数据一：汽车车型、配置与零部件
  - 数据二：总装线生产工序与状态
  - 数据三：下线检测数据
  - 数据四：运行中的故障诊断码数据
  - 数据五：汽车运行的启动、停止数据
  - 数据六：客服通话
  - 数据七：道路救援
  - 数据八：维修数据

讨论

- **假如希望开发故障预诊断功能，将来通过OTA更新**

  - 数据一：汽车车型、配置与零部件
  - 数据二：总装线生产工序与状态
  - 数据三：下线检测数据
  - 数据四：运行中的故障诊断码数据
  - 数据五：汽车运行的启动、停止数据
  - 数据六：客服通话
  - 数据七：道路救援
  - 数据八：维修数据

或许能提供一些标签

讨论

- **假如希望开发故障预诊断功能，将来通过OTA更新**

- 数据一：汽车车型、配置与零部件
- 数据二：总装线生产工序与状态
- 数据三：下线检测数据
- 数据四：运行中的故障诊断码数据
- 数据五：汽车运行的启动、停止数据
- 数据六：客服通话
- 数据七：道路救援
- 数据八：维修数据

或许能直接作为特征

处理后可以作为特征

讨论

- 数据四：运行中的故障诊断码数据
- 数据五：汽车运行的启动、停止数据

- **特征示例**
  - 故障前总里程数，故障前发动机总启停次数
  - 故障前发动机停机时间（以判断是冷启动还是热启动）
  - 故障前一个月发动机总启停次数，总运行时间
  - 故障前一个月/一周 XXX 故障诊断码是否出现，出现次数
  - YYY 故障诊断码在发动机启动后 mm 分钟内出现的次数
  - ……

讨论

- **特征如何提取**
  - 来自观察与数据挖掘
  - 来自专业领域知识（domain knowledge）